# Assignment - 4

# Problem 1:

| S.No. | Line No. | Error Description | Error Type | Error Correction |
|---|---|---|---|---|
| **1** | 10 | Semicolon `;` is missing at the end of `int department = 1` | Syntax Error | `int department = 1;` |
| **2** | 12 | A string `"90"` is assigned to an integer variable `marks`. | Type Mismatch | `int marks = 90;` |
| **3** | 27 | `if(x = 10)` uses an assignment operator instead of a comparison operator. | Logical Error | `if(x == 10)` |
| **4** | 32 | The variable `score` is printed without being declared first. | Compiler Error | Declare it before use: `int score = 100;` |
| **5** | 34 | Re-declaring `totalStudents` inside `main` shadows the global variable. | Variable Shadowing | Rename the local one: `int localStudents = 100;` |
| **6** | 50 | The function `add(a, b)` is called without a prototype declaration. | Linker Error | Add a prototype at the top: `int add(int, int);` |
| **7** | 58 | The variable `temp` is accessed outside the block where it was declared. | Scope Error | Declare `int temp;` outside the curly braces. |
| **8** | 62 | Accessing index 10 of `arr` when the array size is only 5. | Runtime Error | Use a valid index (0 to 4): `arr[4] = 100;` |
| **9** | 66 | Format specifier `%d` is incorrectly used to print a `float` variable `salary`. | Format Mismatch | Use `%f`: `printf("%f\n", salary);` |
| **10** | 71 | Re-declaring `int count = 100;` right after updating it creates a scope conflict. | Shadowing Error | Rename the inner variable: `int innerCount = 100;` |
| **11** | 86 | The variable `result` is declared inside the loop block but accessed outside. | Scope Error | Declare `int result;` before starting the loop. |
| **12** | 91-94 | Both the `if` and `else` blocks execute the exact same print statement. | Logical Error | Change the else block: `else printf("Not Eligible\n");` |

| | | | | |
|---|---|---|---|---|
| **13** | 98 | The function `displayData()` is called without passing the required argument. | Parameter Mismatch | Pass the variable: `displayData(value);` |
| **14** | 130 | Semicolon after `while(z > 0);` creates an infinite loop that halts execution. | Logical Error | Remove the semicolon: `while(z > 0)` |
| **15** | 135 | Double quotes `"A"` signify a string, which cannot fit into a `char` variable. | Type Mismatch | Use single quotes: `char grade = 'A';` |
| **16** | 139 | Format specifier `%s` is used to print an integer variable `number`. | Runtime Error | Use `%d`: `printf("%d\n", number);` |
| **17** | 143 | Dereferencing a NULL pointer (`*ptr = 100`) causes an immediate crash. | Segmentation Fault | Point it to a valid variable address first: `ptr = &number;` |

# Problem 2:

| S.No. | Line | Error Description | Error Type | Error Correction |
|---|---|---|---|---|
| **1** | 10 | A string `"90"` is assigned to an integer variable `marks`. | Type Mismatch | `int marks = 90;` |
| **2** | 16 | Semicolon `;` is missing at the end of the `printf` statement. | Syntax Error | `printf("Program Started\n");` |
| **3** | 24 | The `extern int total` variable is printed without a real definition. | Linker Error | Define `int total = 0;` in the global scope. |
| **4** | 35 | The function `add(10, 20)` is called without a prototype declaration. | Prototype Error | Add a prototype at the top: `int add(int, int);` |
| **5** | 41 | Trying to fetch the address (`&number`) of a `register` variable is illegal. | Compiler Error | Remove the `register` keyword to read its memory address. |
| **6** | 43 | A string `"50000"` is assigned to a static integer variable `salary`. | Type Mismatch | Change to an integer: `static int salary = 50000;` |

| | | | | |
|---|---|---|---|---|
| **7** | 51 | The variable `department` is accessed outside its local block scope. | Scope Error | Move the declaration outside the inner curly braces. |
| **8** | 60 | The variable `temp` is declared inside a loop but printed outside it. | Scope Error | Declare `int temp;` before the loop begins. |
| **9** | 64 | Writing to index 10 of `arr` when the maximum size allocated is 5. | Runtime Error | Use a valid index within bounds (0 to 4). |
| **10** | 69 | Dividing a number by zero (`a / b` where `b=0`) crashes the program. | Runtime Error | Ensure `b` is not zero before executing division. |
| **11** | 73 | `if(value = 100)` executes an assignment instead of a comparison. | Logical Error | Change to comparison: `if(value == 100)` |
| **12** | 80-83 | Both `if` and `else` blocks print out the exact same message "Pass". | Logical Error | Change the else block: `else printf("Fail\n");` |
| **13** | 91 | Double quotes `"A"` represent a string and cannot be stored in a `char`. | Type Mismatch | Use single quotes: `char grade = 'A';` |
| **14** | 120 | Printing an uninitialized variable `data` outputs random garbage values. | Logical Error | Initialize it during declaration: `int data = 0;` |
| **15** | 124 | Modifying a `NULL` pointer target (`*ptr = 100`) crashes the runtime. | Segmentation Fault | Assign a proper variable reference to the pointer. |
| **16** | 126 | `extern int companyCode` is declared but never defined anywhere. | Linker Error | Add a global definition: `int companyCode = 101;` |
| **17** | 132 | `calculateAverage` is called without a valid prototype definition. | Compiler Error | Declare `int calculateAverage(int, int);` at the top. |
| **18** | 136 | A string literal `"EMP101"` is directly assigned to an integer `employeeId`. | Type Mismatch | Change type to a string pointer: `char *employeeId = "EMP101";` |
| **19** | 138 | Printing a string-loaded integer with format specifier `%d` fails. | Format Mismatch | Use `%s` along with a proper string datatype. |
| **20** | 140 | Semicolon after `while(counter > 0);` locks the code into an infinite loop. | Logical Error | Remove the trailing semicolon. |

# Problem 3:

| S.No. | Line No. | Error Description | Error Type | Error Correction |
|---|---|---|---|---|
| **1** | 22 | Semicolon `;` is missing at the end of `int choice`. | Syntax Error | `int choice;` |
| **2** | 56 | Colon `:` is missing after the default keyword switch block. | Syntax Error | `default:` |
| **3** | 85 | The bound check `index > 5` happens after array insertion, causing a crash. | Runtime Error | Validate bounds before writing data: `if(index >= 5)` |
| **4** | 94 | The loop condition `i <= totalStudents` accesses an uninitialized array index. | Logical Error | Use a strict upper limit: `for(i=0; i<totalStudents; i++)` |
| **5** | 120 | The math operation `total / 0` triggers an immediate runtime execution crash. | Runtime Error | Change denominator to count variable: `return total / totalStudents;` |
| **6** | 126 | `highest` and `lowest` are typed as `int` but receive `float` marks values. | Type Mismatch | Declare them as floats: `float highest = students[0].marks;` |
| **7** | 144 | Format specifier `%d` is used to print out a floating-point highest score. | Format Mismatch | Use float specifier: `printf("Highest = %.2f\n", highest);` |
| **8** | 146 | Format specifier `%d` is used to print out a floating-point lowest score. | Format Mismatch | Use float specifier: `printf("Lowest = %.2f\n", lowest);` |
| **9** | 149 | Typo in variable name; typed `totalStudent` instead of `totalStudents`. | Compiler Error | Change variable reference name to `totalStudents`. |
| **10** | 151 | The function `printGrade()` is called but is completely missing from the code. | Linker Error | Build and declare a `void printGrade();` function. |
| **11** | 153 | The function `calculateRank()` is called but is completely missing from the code. | Linker Error | Build and declare a `void calculateRank();` function. |

| | | | | |
|---|---|---|---|---|
| **12** | 160 | Storing a double-quoted string `"101"` inside an integer field `s.rollNo`. | Type Mismatch | Remove quotes to store as a number: `s.rollNo = 101;` |
| **13** | 164 | Storing a double-quoted string `"90"` inside a floating-point field `s.marks`. | Type Mismatch | Store it directly as a decimal: `s.marks = 90.0;` |
| **14** | 179 | Attempting to assign `NULL` directly to an entire struct block instance. | Compiler Error | Clear memory via `memset` or handle with a validation flag. |
| **15** | 188 | Comparing string arrays using the raw relational equality operator `==`. | Logical Error | Swap with safe comparison function: `if(strcmp(name, students[0].name) == 0)` |
| **16** | 199 | Implicitly converting a float-returning function output into an integer `avg`. | Type Mismatch | Retain precision by changing data type to `float avg;` |
| **17** | 201 | Printing float properties with an integer format specifier `%d`. | Format Mismatch | Switch to decimal-point printing: `printf("%.2f\n", avg);` |
| **18** | 215 | Opening a target data file in read-only mode `"r"` then calling `fprintf`. | I/O Error | Modify file system access flag to write mode: `"w"` |
| **19** | 226 | Calling `fscanf` on a file stream `fp` without running a validation test first. | Runtime Error | Add safety validation wrapper: `if (fp != NULL)` |

# Problem 4:

| S.No. | Line No. | Error Description | Error Type | Error Correction |
|---|---|---|---|---|
| **1** | 81 | `totalAccounts > 10` boundary verification happens after array memory is already written. | Runtime Error | Run a preventive check beforehand: `if(totalAccounts >= 10)` |
| **2** | 90 | Iterating with `i <= totalAccounts` forces the system to pull garbage field entries. | Logical Error | Restrict processing limits: `for (i=0; i<totalAccounts; i++)` |
| **3** | 115 | Using raw user-input account numbers as direct structural array indices. | Logical Error | Build a scanning loop to evaluate true index positions. |

| | | | | |
|---|---|---|---|---|
| **4** | 134 | The conditional operator check `balance > amount` inside withdrawal logic is reversed. | Logical Error | Invert validation argument: `if(accounts[acc].balance < amount)` |
| **5** | 138 | The `else` block executes processing logic when funds are insufficient. | Logical Error | Swap operational blocks for correct conditional flow. |
| **6** | 185 | Executing division operations using zero counters (`total / totalAccounts`). | Runtime Error | Guard math logic with conditions: `if(totalAccounts > 0)` |
| **7** | 194 | Compiler cannot resolve `totalAccount` because the real variable ends with an 's'. | Compiler Error | Correct the reference name spelling to `totalAccounts`. |
| **8** | 196 | Invoking a missing operational routine `calculateInterest()` without declaration. | Linker Error | Prototype and design a functional `void calculateInterest();` block. |
| **9** | 204 | Placing a text value string `"1001"` directly inside an integer type field. | Type Mismatch | Provide clean unquoted integers: `a.accNo = 1001;` |
| **10** | 208 | Placing a text value string `"50000"` directly inside a floating-point variable. | Type Mismatch | Provide a float data value format: `a.balance = 50000.0;` |
| **11** | 224 | Directly assigning a structural record element block location to a `NULL` literal value. | Compiler Error | Use clean structure resets or assign specific system status flags. |
| **12** | 235 | Verifying content equality across string structures with basic `==` relational operators. | Logical Error | Implement string library utilities: `if(strcmp(name, accounts[0].name) == 0)` |
| **13** | 244 | Calling an unmapped, undefined data method named `generateAverage()`. | Compiler Error | Declare and map out the average calculation function. |
| **14** | 253 | Accessing disk storage via read-only `"r"` rules while attempting data exports. | I/O Error | Open streams using output parameters: `fopen("accounts.txt", "w");` |
| **15** | 264 | Processing variables via `fscanf` streams without evaluating the existence of files. | Runtime Error | Build safety checks into file routines: `if (fp != NULL)` |
| **16** | 299 | Attempting out-of-bounds writes into position 10 of a 5-element array. | Runtime Error | Restrict assignment actions to legal indices (0 through 4). |

| | | | | |
|---|---|---|---|---|
| **17** | 301 | Forcing value writes directly into unmapped space (`NULL` address location pointer). | Segmentation Fault | Direct pointer values to legitimate structures before updates. |
| **18** | 306 | Executing hardcoded division algorithms with explicit zero value. | Runtime Error | Use non-zero denominators to preserve system operation stability. |

# Program 5:

| S.No. | Line No. | Error Description | Error Type | Error Correction |
|---|---|---|---|---|
| **1** | 25 | Semicolon `;` is missing at the end of the `int choice` declaration. | Syntax Error | Change to `int choice;` |
| **2** | 72 | Colon `:` is missing after the `default` keyword inside the switch case. | Syntax Error | Change to `default:` |
| **3** | 98 | The array capacity check `if(totalBooks > 20)` occurs *after* writing data. At index 20, it triggers an out-of-bounds crash. | Logical/ Runtime Error | Check array limits before data insertion: `if(totalBooks >= 20)` |
| **4** | 109 | The loop condition `i <= totalBooks` makes the program read one uninitialized empty structure slot past the active boundary. | Logical Error | Restrict matching boundaries: `for(i=0; i<totalBooks; i++)` |
| **5** | 127, 147 | The user-input `id` value is directly mapped as a structural array index. If book IDs are sequential (e.g., 100, 101), this causes a memory violation. | Logical Error | Implement an active tracking loop to match `books[i].bookId == id;` |
| **6** | 187 | The math statement `totalPrice / totalBooks` initiates a division process that crashes the runtime if there are zero books. | Runtime Error (Divide by Zero) | Use safety validation blocks: `if(totalBooks > 0)` |
| **7** | 191 | The spelling references `totalBook` instead of the globally managed variable name `totalBooks`. | Compiler Error | Modify spelling reference to `totalBooks`. |
| **8** | 195 | The operational routine `calculateFine()` is triggered without | Linker Error | Declare and map out a matching `void` |

| | | | | |
|---|---|---|---|---|
| | | a declaration prototype or definition block. | | `calculateFine();` block. |
| **9** | 202 | A text string literal `"101"` is assigned directly to an explicit integer structure member `b.bookId`. | Type Mismatch | Change to raw numeric initialization: `b.bookId = 101;` |
| **10** | 208 | A text string literal `"500"` is assigned directly to a floating-point destination member `b.price`. | Type Mismatch | Supply numeric literal types: `b.price = 500.0;` |
| **11** | 220 | Using raw relational equality operators `==` to track matching strings only evaluates memory addresses instead of characters. | Logical Error | Implement string utility operations: `if(strcmp(title, books[i].title) == 0)` |
| **12** | 233 | Attempting to clear structure records by explicitly setting the array position index address directly to `NULL`. | Compiler Error | Use clean tracking data resets or define active status tracking flags. |
| **13** | 250 | Triggering an unmapped, completely missing functional sub-routine named `calculateAveragePrice()`. | Compiler Error | Introduce proper tracking prototypes at the top of your module file. |
| **14** | 260 | Opening file streams inside a read-only parameter mode `"r"` while attempting to apply raw data write operations via `fprintf`. | I/O / Logical Error | Change file opening parameters to output settings: `"w"` |
| **15** | 271 | Accessing files using `fscanf` reading routines without testing whether the targeted stream address point `fp` is successfully open. | Runtime Error | Implement file validation workflows: `if (fp != NULL)` |
| **16** | 304 | Attempting to forcefully pass values straight into an unmapped space via an explicit `NULL` pointer target initialization `(*ptr = 100)`. | Segmentation Fault | Assign legitimate operational memory addresses to pointers prior to value allocation. |
| **17** | 308 | Overwriting array assignment registers at index position 10 when the target data variable size properties only span 5 slots. | Runtime Error (Buffer Overflow) | Safely limit indices to allocated ranges (positions 0 through 4). |
| **18** | 314 | Calculating standard math output steps using an absolute zero value denominator (`a / b` where `b=0`). | Runtime Error | Check denominator arguments before triggering execution operations. |
| **19** | 322 | Forcing a large text payload "Programming" straight into an explicit character array variable string allocation destination size of 5. | Runtime Error (Buffer Overflow) | Upgrade character array bounds minimum limits to 12. |

| | | | | |
|---|---|---|---|---|
| **20** | 332 | Allocating explicit memory pools using `malloc(5)` properties, yet iterating up to index 20 inside loop counters, destroying heap tracking layouts. | Runtime Error | Set loops to align with matching dynamic block limits: `for(int i=0; i<5; i++)` |
| **21** | 342 | Reading value properties inside print routines from pointer target allocations that were already destroyed via memory execution states. | Use After Free | Execute value print calls prior to invoking allocation cleanup commands. |
| **22** | 339, 344 | Handing over identical dually executed allocation reference destinations (`ptr`) to multiple clean-up commands. | Runtime Error (Double Free) | Erase additional repetitive dynamic deallocation statements. |